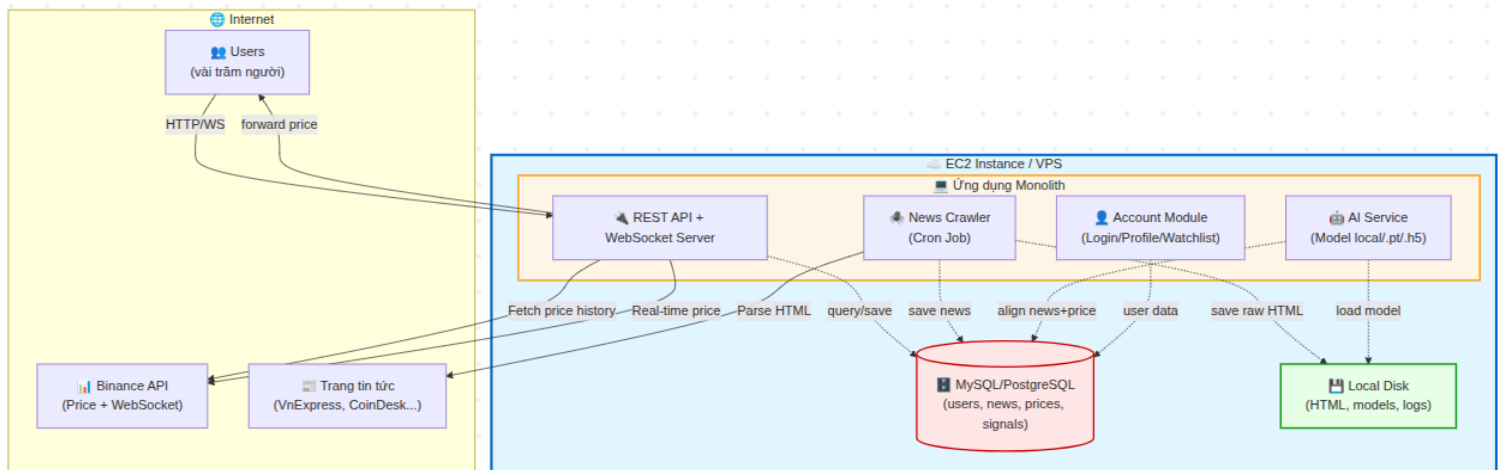


Họ và tên: Võ Phi Hổ
MSSV: 22120106

Phiên bản 1 – Single Box Monolith (ít user, dễ làm đồ án)



Giả định:

- Vài chục – vài trăm user.
- Chỉ cần chạy demo ổn định, chưa cần high availability.
- Độ trễ chấp nhận được.

Kiến trúc: 1 EC2 instance (hoặc 1 VPS) duy nhất trong VPC:

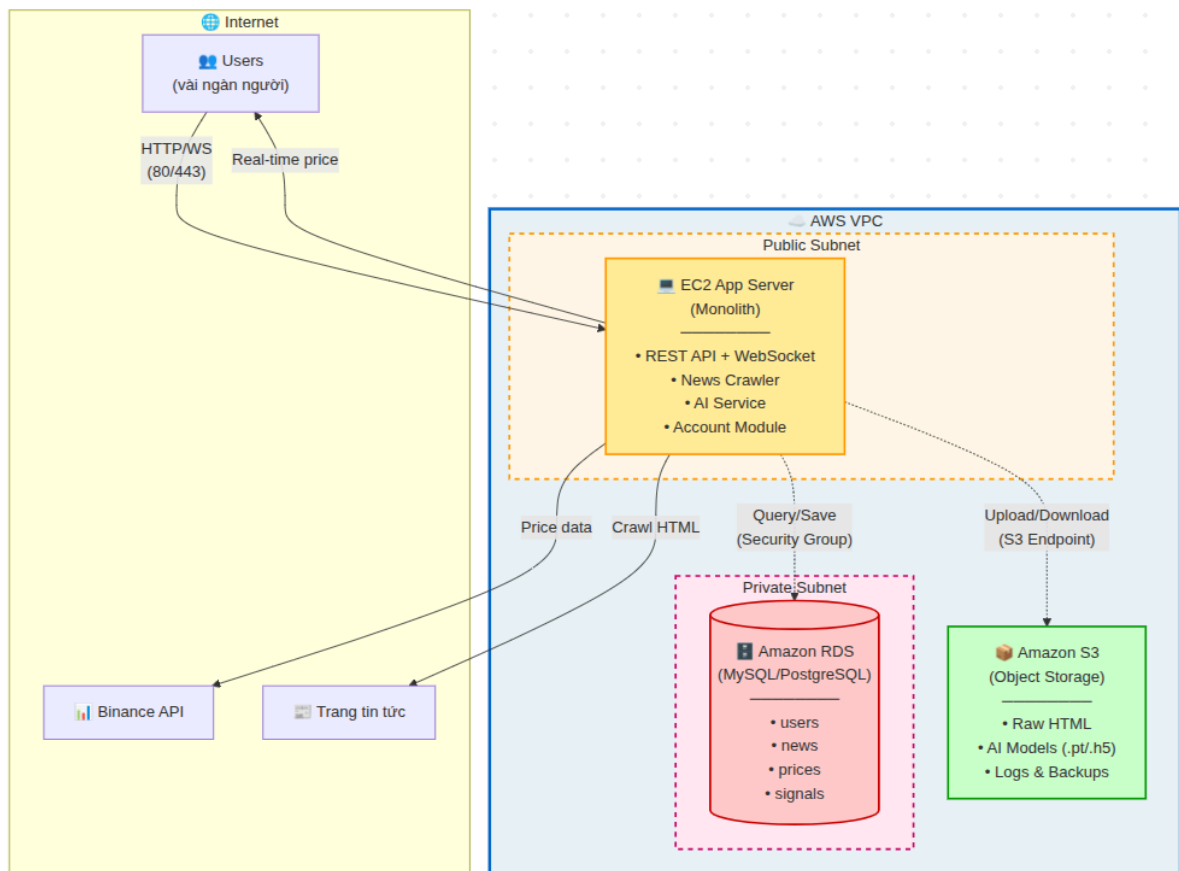
- Web Server + Ứng dụng Monolith:
 - REST API + WebSocket Server:
 - Lấy giá từ Binance API (REST) để build lịch sử.
 - Mở WebSocket đến Binance để nhận giá realtime và forward ra cho client.
 - Crawler tin tức:
 - Cron job/Background task (vd: Celery/RQ/Sidekiq hoặc đơn giản là cron + script).
 - Gọi HTTP đến nhiều trang báo, parse HTML, trích xuất:
 - Tiêu đề, timestamp, nguồn, link, nội dung, sentiment sơ bộ nếu có.
 - Lưu ý: Mỗi trang có cấu trúc khác nhau → viết adapter / rule cho từng trang.
 - Lưu raw HTML + thông tin đã trích xuất.
 - Service AI:
 - Gọi model AI từ local (file .pt, .h5, ...) hoặc API ngoài (hộp đen).

- Align tin tức với giá lịch sử để tạo feature (vd: 1h sau tin đó giá UP/DOWN).
- Module Account:
 - Đăng ký/Đăng nhập.
 - Profile user, watchlist cập tiền.
 - Lưu config GUI (layout chart, khung thời gian yêu thích).
- MySQL/PostgreSQL cài trên cùng máy:
 - Bảng user, news, prices, signals, logs.
- Storage trên local disk:
 - Lưu raw HTML, file model AI, logs.

Tại sao chọn kiến trúc này?

- Giống bước **Start with a single box** trong Scaling AWS:
 - Đơn giản, dễ code, dễ deploy, phù hợp giai đoạn đồ án đầu.
 - Tất cả nằm chung 1 chỗ → debug nhanh, ít network hop.
- Hạn chế:
 - Không scale ngang được (horizontal) → chỉ scale dọc bằng cách nâng máy (CPU/RAM/disk).
 - Không có high availability: máy down là hệ thống chết.
 - Crawler, WebSocket, AI, GUI chia sẻ tài nguyên → dễ nghẽn CPU/RAM/IO.

Phiên bản 2 – Users+ : Tách Database + Object Storage



Khi số lượng user tăng (vài ngàn), crawler lấy tin từ nhiều nguồn hơn, dữ liệu tin tức và giá lịch sử tăng rất nhanh.

Kiến trúc

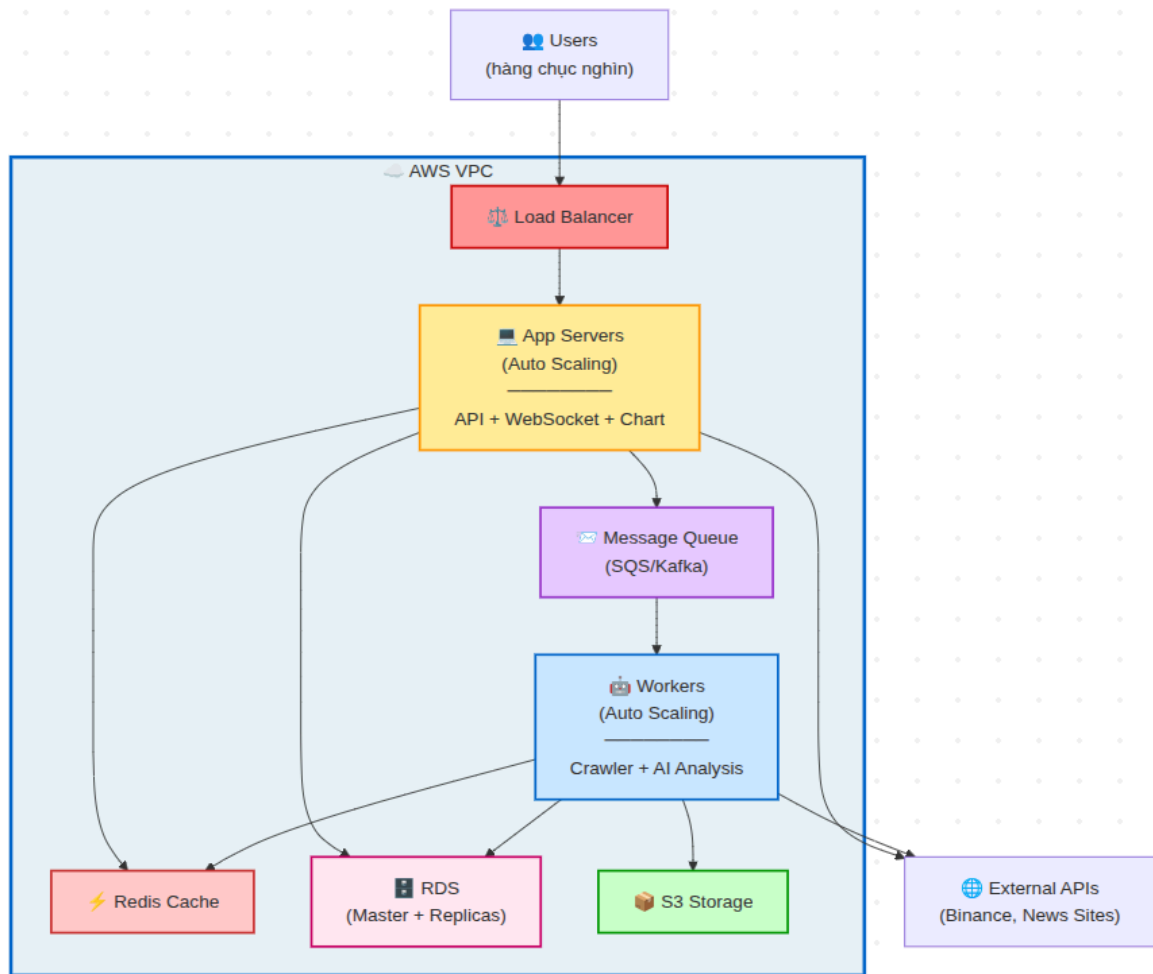
- EC2 App Server (chạy monolith như cũ, nhưng bỏ DB ra ngoài):
 - Web + API + WebSocket + Crawler + AI (vẫn là 1 codebase).
- RDS (MySQL/PostgreSQL):
 - Tách DB ra thành máy riêng (managed service).
 - Bảng tin tức, giá, user, signals.
- S3 (Object Store):
 - Lưu:
 - Raw HTML của news (để re-parse khi site đổi HTML).
 - File model AI.
 - Log lớn, file backup.
- VPC:
 - App server ở public subnet.
 - RDS + S3 access ở private (qua endpoint, security group).

- Security:
 - Security group chỉ cho App server connect đến RDS.
 - Port mở ra internet: 80/443.

Ý nghĩa thay đổi

- Bám theo step “Users+”: Tách web server và database + dùng Object Store như tài liệu AWS scaling.
- Lợi ích:
 - DB có thể scale riêng (tăng RAM, IOPS).
 - S3 xử lý tốt khối lượng lớn file tin tức & raw HTML (1 TB/tháng là bình thường).
 - Ứng dụng nhẹ hơn, không lo full disk vì news/history.
- Hạn chế:
 - Vẫn chỉ có 1 app server → dễ nghẽn khi WebSocket connection nhiều, crawler chạy mạnh, AI chạy nặng.

Phiên bản 3 – Users++ : Load Balancer + Scale ngang App + Worker Crawler/AI



Bắt đầu có:

- Hàng chục nghìn user.
- Biểu đồ realtime cho nhiều cặp tiền, đa khung thời gian.
- Crawler tin từ nhiều site, AI phân tích nặng hơn.

Kiến trúc

- ALB (Application Load Balancer):
 - Nhận HTTP/HTTPS + WebSocket từ client.
 - Phân phối request đến nhiều App server.
 - Đây là pattern “Horizontal scaling web servers” trong Scaling AWS.
- App Server Auto Scaling Group:
 - Nhiều EC2 instance chạy:
 - API cho GUI (lấy news đã xử lý, giá, signal).
 - WebSocket để push giá realtime & tín hiệu AI.
 - Render chart (TradingView-like).

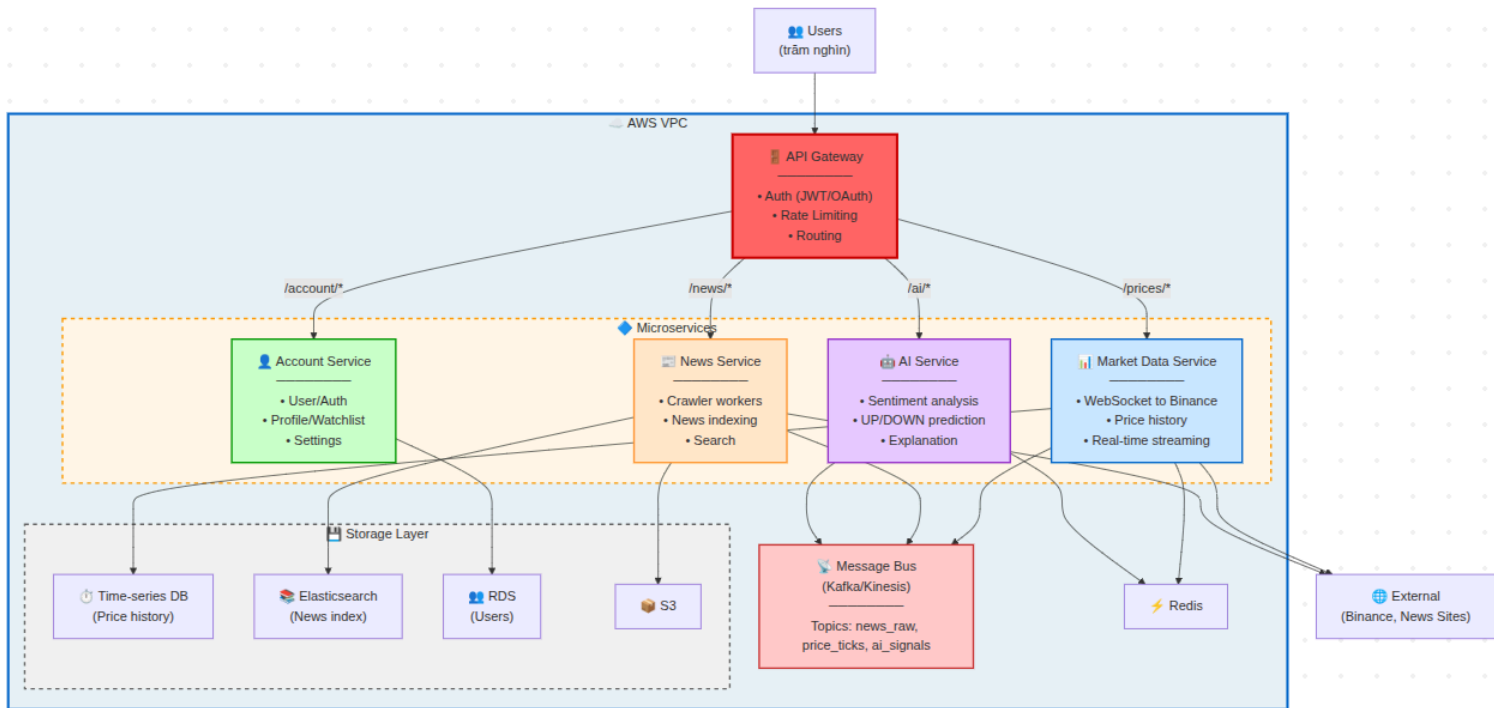
- Không chạy crawler & jobs nặng AI (để tránh block).
- Worker Cluster (Crawler + AI):
 - Một Auto Scaling Group khác, role “Worker”:
 - Crawler Service:
 - Gửi HTTP đến các site news.
 - Học/nhận biết structure:
 - Ban đầu: rule-based theo CSS selector/XPath cho từng site.
 - Nâng cao: ML model phân loại block (title, body, author,...), giúp tự “học” cấu trúc mới, giảm lỗi khi HTML đổi.
 - Đẩy kết quả vào DB + lưu raw HTML lên S3.
 - AI Service:
 - Consumes news + price history.
 - Gọi model AI (hộp đen) để tạo sentiment, dự đoán xu hướng giờ/ngày kế tiếp (UP/DOWN) kèm lý do.
 - Lưu signals vào DB.
- Message Queue (SQS/Kafka):
 - News Fetch Queue: app tạo job khi cần crawl nguồn mới.
 - AI Analysis Queue: crawler xong → đẩy job để AI xử lý.
 - Bám theo pattern “Queues + Workers” trong phần Asynchronism của system-design-primer.
- RDS (master + read replica):
 - Master: ghi.
 - 1–2 read replica: phục vụ query đọc nặng (news list, history chart).
- Cache (Redis/ElastiCache):
 - Cache:
 - Giá hiện tại của BTCUSDT, ETHUSDT, ...
 - Top news mới nhất.
 - Tín hiệu AI nóng.
 - Cache-aside pattern (tham khảo phần Caching ở system-design-primer).

Tại sao cần bước này?

- Nhu cầu người dùng tăng:
 - Xem đa cặp tiền, đa khung thời gian, realtime.
 - Hàng ngàn WebSocket connection → cần nhiều app server.
- Công việc nặng (crawler, AI) cần tách ra:
 - Nếu để cùng app server sẽ làm web bị lag/delay.
 - Queue + Worker giúp xử lý bất đồng bộ, có khả năng backpressure.

- Đáp ứng yêu cầu availability:
 - ALB + nhiều instance + read replica DB → nếu 1 node chết vẫn chạy được.
- Liên hệ với tài liệu:
 - Đây là giai đoạn “Users++”: Horizontal scaling web servers, caching, SQL read replicas, queues (link).

Phiên bản 4 – Users+++ : Microservices nhẹ + API Gateway + Tách domain



Giả sử:

- Lên tới trăm nghìn user, yêu cầu tính modular để đội dev chia nhau làm.
- Cần tăng độ tin cậy từng module và scale độc lập.

Kiến trúc logic (microservices theo domain)

Tham khảo thêm khái niệm reverse proxy / API gateway / load balancer từ bài viết: <https://blog.algomaster.io/p/load-balancer-vs-reverse-proxy-vs-api-gateway>

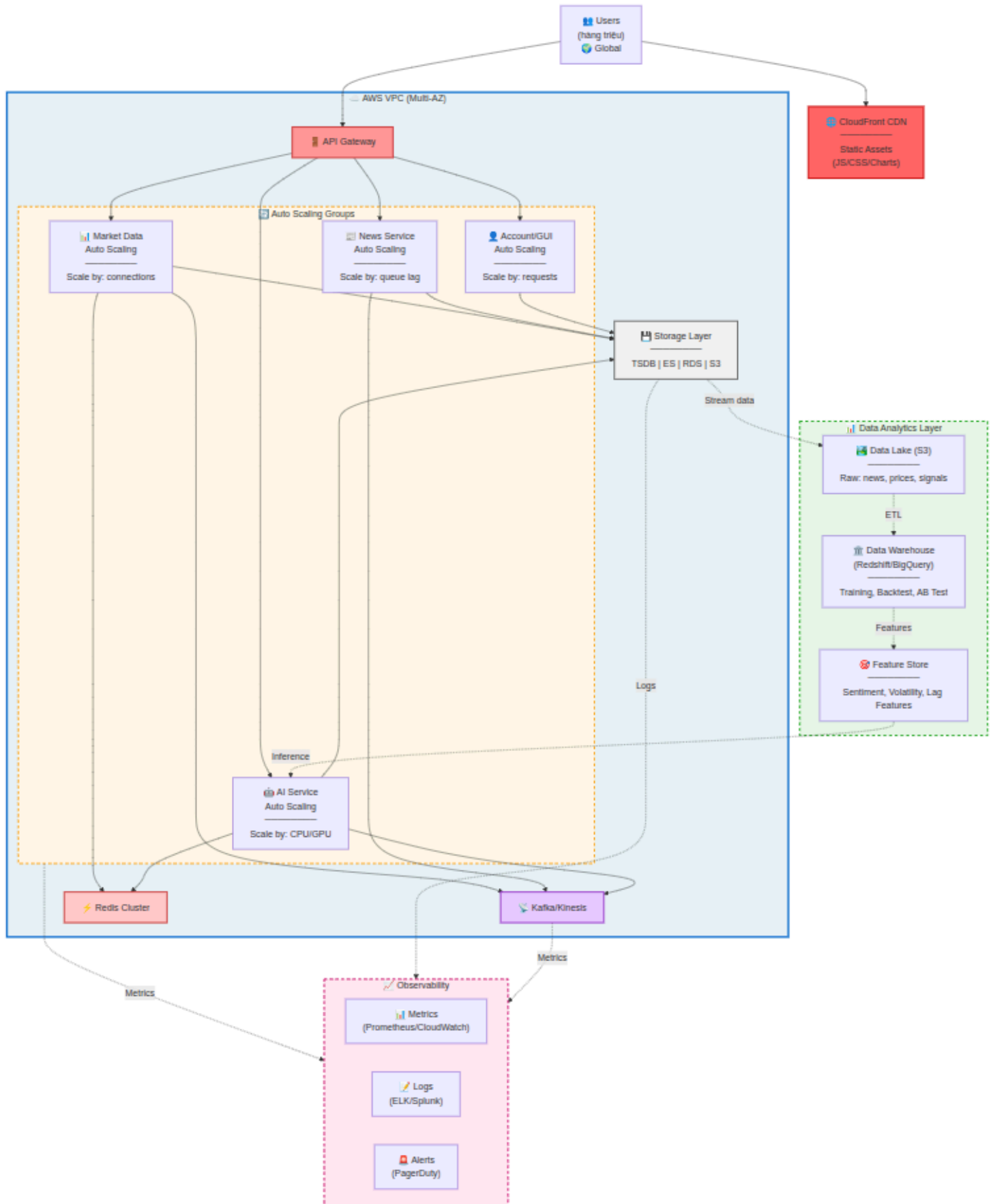
- API Gateway (hoặc Nginx/Envoy + custom):
 - Entry point duy nhất cho client:
 - /news/* → News Service.
 - /prices/* → Market Data Service.
 - /ai/* → AI Service.
 - /account/* → Account Service.
 - Thực hiện:
 - AuthN/AuthZ (JWT, OAuth).
 - Rate limiting, request logging.
- Market Data Service:
 - Kết nối WebSocket đến Binance, duy trì stream giá realtime cho nhiều cặp.
 - Lưu giá lịch sử vào Time-series DB hoặc RDS riêng.

- Expose:
 - REST: lịch sử giá, multi-timeframe (1m, 5m, 1h, 1d).
 - WebSocket/Server-Sent Events: giá hiện tại & candlestick realtime.
- News Service:
 - Quản lý crawler worker.
 - Lưu, index tin tức: title, nội dung, tags, sentiment,... trong DB riêng (có thể dùng Elasticsearch/OpenSearch để search nhanh).
- AI Service:
 - Nhận tin tức đã align với giá, trả về:
 - Sentiment score.
 - Prediction (UP/DOWN) cho giờ/ngày tiếp theo.
 - Explanation (lý do: positive news, high volume, ...).
 - Có thể wrap 1 model hộp đen (HuggingFace, SageMaker endpoint,...).
- Account Service:
 - User, auth, profile, watchlist, setting GUI.
 - Token-based auth (JWT) để các service khác verify.
- Message Bus (Kafka/Kinesis):
 - Topic news_raw, news_parsed, price_ticks, ai_signals.
 - Các service subscribe/publish để giảm coupling.

Tại sao chuyển sang microservices?

- Domain rõ ràng:
 - Đồ án có 4 phần tách biệt (news, price, AI, account) → phù hợp chia service.
- Scale độc lập:
 - Market Data có thể cần nhiều CPU & network hơn.
 - AI Service cần GPU/CPU nặng.
 - News Service cần nhiều IO/network cho crawler.
- Độ tin cậy:
 - Nếu AI Service bị lỗi, phần chart + news vẫn chạy (có thể degrade gracefully).
- Liên hệ tài liệu:
 - Đây thuộc nhóm chủ đề microservices, communications, message queues trong system-design-primer.

Phiên bản 5 – Users++++ : Autoscaling, Observability, Data Lake/ML nâng cao



Khi hệ thống tiến đến:

- Hàng triệu user.
- Khối lượng tin tức & giá rất lớn (tương tự con số trong tài liệu: 1B writes/tháng, 100B reads/tháng).

Thành phần bổ sung

- Autoscaling cho từng nhóm:
 - Auto Scaling Group cho:
 - Market Data Service.
 - News Service.
 - AI Service.
 - Account/GUI.
 - Scale theo:
 - CPU, number of connections, message queue lag, custom metric.
 - Pattern này được mô tả trong phần Users++++ (Autoscaling) của Scaling AWS.
- Global CDN:
 - Frontend static (JS/CSS, bundle TradingView-like) qua CloudFront.
 - Giảm tải App server, giảm latency cho user toàn cầu.
- Data Warehouse / Data Lake:
 - Đổ news + price + signals về:
 - Redshift / BigQuery / Snowflake hoặc S3-based data lake.
 - Dùng cho:
 - Training model mới.
 - Phân tích offline (backtest, AB test chiến lược).
- Feature Store cho AI:
 - Lưu các feature đã xử lý (sentiment series, lag features, volume, volatility, ...).
 - Giúp inference nhanh, training reproducible.
- Observability / Monitoring đầy đủ:
 - Metrics:
 - Latency API, error rate, throughput, queue lag, DB load.
 - Logs:
 - Centralized log (ELK, CloudWatch Logs, Splunk).
 - Alerts:
 - PagerDuty/Email khi lỗi 5xx tăng, latency tăng, AI timeout,...

Tại sao cần bước này?

- Scale chi tiết theo từng tải:
 - Giờ cao điểm (US business hours) giá/WS load tăng đột biến → autoscale Market Data.
 - Khi có major news (SEC, ETF, regulation), News + AI bùng nổ → autoscale Worker/AI.
- Tiếp tục đáp ứng yêu cầu “luôn sẵn sàng hoạt động”:
 - Multiple AZs, autoscaling, health check, rollback, canary deployment.
- Phục vụ bài toán AI nâng cao (nhân quả):
 - Cần nhiều dữ liệu lịch sử tin tức & giá để làm causal inference → data lake & warehouse.